

SCALING VALUE FROM DATA

Understanding Data Marts in Modern Analytics Engineering

A Strategic Framework for Business-Ready Data Models



JIMMY PANG

NOV 25, 2025



6



2

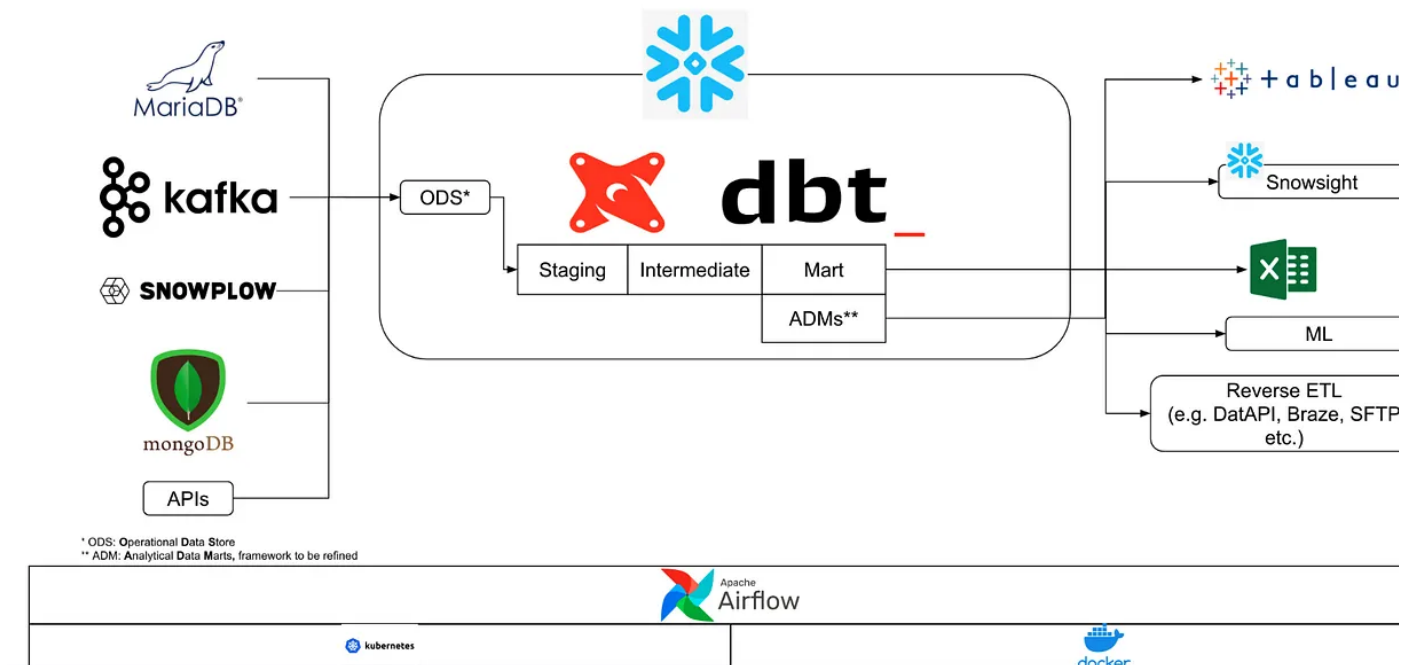
Share



The dbt logo

Data marts represent the culmination of analytics engineering work, the transformation of raw data into business-ready entities that drive decision-making, machine learning, and operational workflow. In modern data architectures, these models serve as the bridge

© 2026 Jimmy Pang · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture



Data Marts act as the **primary** layer for the activation of business (e.g., reporting, analysis, machine-learning & AI, and ReverseETL as well)

Data marts form the **entity layer** where atomic components (staging models) and intermediate transformations converge into coherent business concepts. Each mart represents a distinct entity at its unique grain—an order, customer, territory, click event, or payment transaction. Unlike traditional approaches, modern data warehouses prioritize **denormalization over normalization**. With storage costs decreasing and compute remaining expensive, duplicating data across marts proves more efficient than repeatedly joining concepts during analysis.

This paradigm shift stems from fundamental architectural differences. Traditional Kimball dimensional modeling suited relational database management systems like MySQL, PostgreSQL, and AWS Redshift. Modern OLAP platforms like Snowflake and BigQuery operate as **non-relational systems**, demanding entirely different optimization strategies.

Data Mart Typology

Analytics organizations employ 5 primary mart types, each serving distinct analytical purposes:

1. **Dimension Tables** categorize facts and measures, enabling business question resolution. Standard examples include customer registries, product catalogs, geographic hierarchies, and temporal dimensions. These provide descriptive attributes that contextualize business events.
2. **Fact Tables** capture measurements and metrics from business processes, positioned at the center of analytical schemas. Order transactions, status transitions, and shipment events exemplify fact tables that record what happened in business operations.
3. **Summary Tables** deliver pre-computed, pre-aggregated analytics aligned with specific dimensional cuts and agreed-upon KPIs. These function similarly to semantic layers—centralizing calculation logic while dramatically improving query performance for end users. Building summary tables should leverage existing reporting patterns to ensure alignment with actual consumption needs.
4. **Snapshot Tables** preserve point-in-time states of entities, particularly valuable for slowly changing dimensions. dbt's native snapshot functionality provides the recommended implementation approach for tracking historical attribute changes.
5. **Manual Tables** accommodate data that enters the warehouse through non-standard channels—CSV imports, static reference data, or mapping tables. dbt Seeds offer the preferred mechanism for managing these tables when updates occur infrequently.

Architectural Considerations

Organizations structure data marts through two primary patterns. The **type-based approach** creates dedicated schemas for dimensions, facts, summaries, snapshots, and manual tables—organizing by table characteristics. The **domain-based approach** aligns schemas with business functions like operations, marketing, and finance—organizing by ownership and subject area.

Domain-based structures introduce complexity around organizational changes. Personnel transitions, reorganizations, and scope adjustments can create ownership ambiguity without robust data catalog solutions. This trade-off between business alignment and maintenance overhead requires careful evaluation.

Naming Conventions and Standards

In general, naming convention is a rather organization-specific practice that can vary a lot from one company to another. The following would be the proposed standard.

Effective mart naming follows a hierarchical structure: `<type>_<business_unit>_<domain>_<entity>`. The type prefix (dim, fct, sum, mnl, snp) immediately communicates table purpose and its **ownership**. Business unit identifiers support multi-brand organizations while maintaining consistency. Domain abbreviations (ops, fin, mkt) provide functional categorization. The entity name anchors the convention in the core business concept.

This standardization accelerates onboarding, reduces confusion, and supports automated tooling—creating a self-documenting data environment.

Entity Relationships in Modern Warehousing

The **Entity-Relationship model** remains foundational despite modern platforms abandoning strict relational constraints. Entities represent persistent objects with identity independent of attribute changes—long-lived information users care about, typically database-persisted.

ER modeling employs a top-down approach: identify critical entities and their relationships, characterize these through attributes, then apply constraints. Relationship types define associations between entities, named to describe their function. This methodology provides the conceptual framework for designing coherent, maintainable data marts.

Implementation Patterns

Optimal mart placement within dbt projects evolves with scale. Initial implementations may position marts directly under the models folder. As complexity grows, hierarchical organization becomes essential: `models/marts/{{schema_name}}` provides basic structure, while data mesh architectures demand additional layering, like `models/marts/core/{{schema_name}}` or `models/marts, ops/{{schema_name}}`.

Execution strategies leverage dbt's node selection methods. **Tag-based selection** (`dbt run --select tag:mart_name`) offers superior flexibility compared to path-based approaches. Tags liberate teams from rigid directory structures, enabling logical file organization independent of execution scheduling.

Design Principles and Anti-Patterns

This section highly depends on the exact technology being employed in the organization. It still makes sense to keep Dimensional Modelling for classic Relational databases like PostgreSQL, MySQL & MariaDB, AWS Redshift etc.. But for Non-relational databases like Google BigQuery, MongoDB, AWS DynamoDB etc. would be benefited by de-normalization.

Prioritize denormalized fact tables over dimensional decomposition

Analysts gain efficiency when commonly-needed attributes reside directly in fact tables—eliminating repetitive joins with date and geography dimensions during analysis. This principle directly supports faster insight delivery and improved data consumer experience.

Materialize marts as **tables or incremental models** rather than views. Building physical data structures delivers superior end-user performance and reduces compute costs by avoiding repeated transformation execution. Begin with full-refresh table materializations, introducing incremental complexity only when data volumes and transformation costs justify the added maintenance burden.

Design marts as **wide and denormalized** to minimize downstream join requirements. Modern cloud warehouses make this economically viable—storage costs pale compared to compute expenses. Pack comprehensive entity information into single tables wherever feasible.

Group marts by business domain rather than technical characteristics when organizational scale demands a folder structure. Avoid premature optimization with fewer than 10 marts. Use clear entity names without temporal qualifiers—orders rather

than `orders_per_day`—as time dimensions belong in metric layers.

Thoughtfully build marts upon other marts when computational efficiency demands it. Aggregating order data within customer mart by referencing the orders mart exemplifies appropriate cross-mart dependencies. This pattern leverages previously-built compute and storage investments, though it requires careful DAG management to prevent circular dependencies.

Never create department-specific variations of the same concept. `finance_orders` and `marketing_orders` represent anti-patterns that fragment single sources of truth. When legitimate differences exist—such as tax reporting versus operational revenue measurement—clearly distinguish the concepts themselves: `tax_revenue` versus `revenue`, not `finance_revenue` versus `marketing_revenue`.

Avoid service-specific or geography-specific marts that fragment coherent entities. Resist building tables with excessive join complexity, as intricate dependencies increase data quality risks like duplication.

Operational Considerations

Troubleshooting complex DAGs benefits from temporary materialization changes. When errors surface in late-stage marts but originate in upstream dependencies, converting intermediate models from views to tables localizes error identification. This debugging technique pinpoints failures to their actual source rather than where symptoms appear.

Beyond marts lies the **activation layer**—models that consume marts power specific workflows like reverse ETL operations, machine learning features, or specialized reporting. Marts function both

direct analytical interfaces and as components for activation layer construction, bridging descriptive analytics and operational deployment.

Strategic Implications for Data Leaders

Data marts crystallize data strategy into tangible business value. Their design reflects organizational priorities, analytical maturity, and technical sophistication. Leaders should evaluate mart architectures against several criteria:

Business alignment: Do mart structures mirror how the organization thinks about its operations? Domain-based organization can strengthen data-business partnerships, but demands investment in governance tooling.

Performance optimization: Are compute resources allocated efficiently? Aggressive denormalization and strategic pre-aggregation directly impact the total cost of ownership while improving user experience.

Scalability: Will the chosen patterns accommodate growth in data volume, analytical complexity, and organizational scope? Early architectural decisions create path dependencies that become costly to reverse.

Maintainability: Can analytics engineers understand, modify, and extend existing marts without heroic effort? Clear naming conventions, logical organization, and disciplined anti-pattern avoidance compound over time.

The mart layer ultimately determines whether data investments generate commensurate returns. Technical excellence in staging and intermediate modeling means little if business users cannot easily

extract insights. Data marts represent where engineering rigor meets business impact—the decisive layer in analytics engineering value chains.



6 Likes • 2 Restacks

Discussion about this post

Comments Restacks



Write a comment...